📚 جزوه سیستم عامل | فصل ۸: بن‌بست (Deadlock)

📝 بخش ۱ و ۲: مدل سیستم و بن‌بست در برنامه‌های چندنخی

۱. مقدمه: بن‌بست چیست؟ (به زبان ساده)

تصور کنید دو قطار در یک تقاطع به هم می‌رسند. قانون (تگزاس، اوایل قرن ۲۰) می‌گوید: "هر دو باید بایستند و هیچ‌کدام حق حرکت ندارد مگر اینکه آن یکی رفته باشد!" نتیجه؟ هر دو تا ابد منتظر می‌مانند. این یعنی **بن‌بست**.

در دنیای کامپیوتر: وقتی چند **نخ (Thread)** یا **پردازه (Process)** برای منابع محدود رقابت می‌کنند و هرکدام منتظر منبعی هستند که دست دیگری است، و آن دیگری هم منبعش را آزاد نمی‌کند، سیستم قفل می‌شود.

۲. مدل سیستم (System Model)

برای درک بن‌بست، باید اجزای سیستم را بشناسیم:

الف) منابع (Resources)

سیستم تعدادی منبع محدود دارد که بین نخ‌ها توزیع می‌شوند.

- **انواع منابع:**
  - **سخت‌افزاری:** پردازنده (CPU)، حافظه، دستگاه‌های I/O (پرینتر، درایو نوری).
  - **نرم‌افزاری:** ابزارهای همگام‌سازی مثل Mutex Locks، Semaphores و فایل‌ها.
- **نمونه‌ها (Instances):**
  - هر نوع منبع ممکن است چند "نمونه" داشته باشد.
  - *مثال:* اگر سیستم ۴ پرینتر یکسان داشته باشد، نوع منبع "پرینتر" دارای ۴ نمونه است.
  - *نکته:* اگر درخواستی برای یک منبع بیاید، تخصیص **هر کدام** از نمونه‌های آزاد آن منبع باید کار را راه بیندازد.

ب) چرخه حیات استفاده از منبع (پروتکل استاندارد)

هر نخ برای استفاده از یک منبع **باید** دقیقاً این ۳ مرحله را طی کند:

1. **درخواست (Request):** نخ درخواست منبع می‌کند. اگر منبع آزاد بود، می‌گیرد. اگر نبود، باید **صبر** کند (Wait) تا آزاد شود.
2. **استفاده (Use):** روی منبع عملیات انجام می‌دهد (مثلاً پرینت گرفتن یا نوشتن در متغیر مشترک).
3. **رهاسازی (Release):** منبع را آزاد می‌کند تا بقیه بتوانند استفاده کنند.

نکته سیستمی: سیستم عامل یک جدول دارد که وضعیت همه منابع (آزاد است یا دست چه کسی است) و صف‌های انتظار را مدیریت می‌کند.

ج) تعریف دقیق بن‌بست (نکته امتحانی ✸)

یک مجموعه از نخ‌ها در وضعیت بن‌بست هستند اگر:

"هر نخ در آن مجموعه، منتظر رویدادی باشد که وقوع آن رویداد فقط توسط نخ دیگری در همان مجموعه امکان‌پذیر است."

۳. بن‌بست در برنامه‌های چندنخی (Multithreaded Deadlock)

در سیستم‌های مدرن، سیستم عامل معمولاً جلوی بن‌بست را نمی‌گیرد؛ این برنامه‌نویسان هستند که با استفاده نادرست از قفل‌ها (Mutex) باعث بن‌بست می‌شوند.

سناریوی ایجاد بن‌بست (مثال کلاسیک)

فرض کنید دو قفل (منبع) داریم: 1M و 2M. و دو نخ داریم: Thread 1 و Thread 2.

نخ اول (1 Thread):

1. قفل 1M را می‌گیرد.
2. (برای ادامه کار) درخواست قفل 2M را می‌دهد.

نخ دوم (2 Thread):

1. قفل 2M را می‌گیرد.
2. (برای ادامه کار) درخواست قفل 1M را می‌دهد.

چه اتفاقی می‌افتد؟

1. Thread 1 منتظر 2M است (که دست Thread 2 است).
2. Thread 2 منتظر 1M است (که دست Thread 1 است).
3. **نتیجه:** هیچکدام منبع خود را رها نمی‌کنند و تا ابد منتظر می‌مانند.

نکته مهم: بن‌بست به **زمان‌بندی (Scheduling)** و سرعت اجرا بستگی دارد. اگر Thread 1 خیلی سریع باشد و هر دو قفل را بگیرد و آزاد کند قبل از اینکه Thread 2 بیدار شود، بن‌بست رخ نمی‌دهد. این "غیرقابل پیش‌بینی بودن"، دیباگ کردن بن‌بست را بسیار دشوار می‌کند.

۴. قفل زنده (Livelock) - فامیلِ پرتحرکِ بن‌بست!

قفل زنده شبیه بن‌بست است، اما یک تفاوت ظاهری بزرگ دارد.

- **تعریف:** در قفل زنده، نخ‌ها مسدود (Block) نشده‌اند؛ آن‌ها مدام در حال تلاش، بررسی وضعیت و تغییر حالت هستند، اما **هیچ پیشرفت واقعی (Progress)** در کارشان حاصل نمی‌شود.
- **مثال دنیای واقعی:** وقتی در پیاده‌رو با کسی شاخ‌به‌شاخ می‌شوید:
  - شما می‌روید راست، او هم می‌آید راست.
  - شما می‌روید چپ، او هم می‌آید چپ.
  - هر دو دارید حرکت می‌کنید (زنده هستید!) اما رد نمی‌شوید.

مثال کدنویسی (استفاده از trylock برای اجتناب از بن‌بست)

برنامه‌نویس برای جلوگیری از بن‌بست کدی می‌نویسد که: "اگر قفل دوم را نتوانستی بگیری، قفل اول را هم ول کن و دوباره از اول تلاش کن".

1. 1T: قفل ۱ را گرفت -> قفل ۲ را خواست (نشد) -> قفل ۱ را ول کرد -> دوباره تلاش.
2. 2T: قفل ۲ را گرفت -> قفل ۱ را خواست (نشد) -> قفل ۲ را ول کرد -> دوباره تلاش.
3. **نتیجه:** این چرخه مدام تکرار می‌شود. CPU به شدت درگیر است، اما هیچ کاری انجام نمی‌شود.

راه‌حل قفل زنده: عقب‌نشینی تصادفی (Ethernet Backoff) 💡

راه‌حل این است که نخ‌ها بعد از شکست، **یک مدت زمان تصادفی (Random)** صبر کنند و بعد دوباره تلاش کنند. این کار باعث می‌شود همزمانی (Synchronization) بین تلاش‌های دو نخ شکسته شود (مثل پروتکل شبکه اترنت).

📊 جدول طلایی مقایسه (مخصوص شب امتحان)

ویژگی	بن‌بست (Deadlock)	قفل زنده (Livelock)
وضعیت نخ‌ها	کاملاً مسدود (Blocked) و در صف انتظار هستند.	در حال اجرا (Running) و تغییر وضعیت مداوم هستند.
مصرف CPU	معمولاً مصرف CPU پایین است (چون نخ‌ها خوابند).	مصرف CPU بالا است (چون مدام تلاش می‌کنند).
پیشرفت کار	صفر (متوقف).	صفر (درجا زدن).
حس ناظر بیرونی	برنامه هنگ کرده است.	برنامه گیر کرده اما CPU دارد کار می‌کند (فن سیستم روشن می‌شود!).
راه‌حل	پیشگیری، اجتناب یا تشخیص توسط سیستم.	استفاده از زمان انتظار تصادفی (Random Backoff).

🌟 نکات کلیدی برای نمره ۲۰

1. **منابع فقط سخت‌افزار نیستند:** فراموش نکنید که قفل‌ها (Mutex) و سمافورها هم منبع حساب می‌شوند و در برنامه‌نویسی مدرن، عامل اصلی بن‌بست هستند.
2. **نقش سیستم عامل:** سیستم عامل‌ها (مثل ویندوز یا لینوکس) معمولاً استراتژی خاصی برای جلوگیری از بن‌بست برنامه‌های کاربردی ندارند. مسئولیت با برنامه‌نویس است.
3. **شرایط رقابتی:** بن‌بست معمولاً ناشی از یک نوع شرایط رقابتی (Race Condition) بر سر منابع است.